

Pesquisa sobre Bibliotecas Python para Conexão com Bancos de Dados

1. pyodbc

1. Objetivo principal: Permitir que aplicações Python se conectem a bancos de dados através do padrão ODBC (Open Database Connectivity), um driver universal que funciona com múltiplos SGBDs.

2. Tipos de banco suportados: SQL Server, MySQL, PostgreSQL, Oracle, Access, e qualquer banco que tenha um driver ODBC disponível.

3. Relacional ou não relacional: Relacional.

4. SQL puro, ORM ou ambos: Apenas SQL puro.

5. Instalação:

```
bash
pip install pyodbc
```

6. Exemplo de conexão:

```
python
import pyodbc

conn = pyodbc.connect(
'DRIVER={ODBC Driver 17 for SQL Server};'
'SERVER=localhost;'
'DATABASE=minhabase;'
'UID=usuario;'
'PWD=senha'
)
```

7. Consulta SELECT:

```
python
cursor = conn.cursor()
cursor.execute("SELECT * FROM clientes")

for linha in cursor.fetchall():
    print(linha)

conn.close()
```

2. pymssql

1. Objetivo principal: Fornecer uma interface simples e direta para conexão especificamente com Microsoft SQL Server, sem depender de drivers ODBC instalados no sistema.

2. Tipos de banco suportados: Apenas Microsoft SQL Server.

3. Relacional ou não relacional: Relacional.

4. SQL puro, ORM ou ambos: Apenas SQL puro.

5. Instalação:

```
bash
pip install pymssql
```

6. Exemplo de conexão:

```
python
import pymssql

conn = pymssql.connect(
    server='localhost',
    user='usuario',
    password='senha',
    database='minhabase'
)
```

7. Consulta SELECT:

```
python
cursor = conn.cursor()
cursor.execute("SELECT * FROM clientes")

for linha in cursor.fetchall():
    print(linha)

conn.close()
```

3. psycopg2

1. Objetivo principal: Ser o driver/adaptador mais utilizado para conectar aplicações Python especificamente ao PostgreSQL.

2. Tipos de banco suportados: Apenas PostgreSQL.

3. Relacional ou não relacional: Relacional.

4. SQL puro, ORM ou ambos: Apenas SQL puro.

5. Instalação:

```
bash
pip install psycopg2-binary
```

6. Exemplo de conexão:

```
python
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    dbname="minhabase",
    user="usuario",
    password="senha",
    port="5432"
)
```

7. Consulta SELECT:

```
python
cursor = conn.cursor()
cursor.execute("SELECT * FROM clientes")

for linha in cursor.fetchall():
    print(linha)

conn.close()
```

4. SQLAlchemy

1. Objetivo principal: Ser um toolkit/ORM completo que abstrai o acesso a bancos de dados, permitindo trabalhar com objetos Python em vez de escrever SQL manualmente, além de gerenciar conexões, transações e migrações de forma padronizada.

2. Tipos de banco suportados: PostgreSQL, MySQL, SQLite, SQL Server, Oracle, (funciona como uma camada sobre outros drivers, como psycopg2, pyodbc, etc.).

3. Relacional ou não relacional: Relacional. Não trabalha nativamente com bancos NoSQL.

4. SQL puro, ORM ou ambos: Ambos — oferece o **Core** e o **ORM** (mapeamento de classes Python para tabelas).

5. Instalação:

```
bash
pip install sqlalchemy
# + driver do banco específico, ex:
pip install psycopg2-binary
```

6. Exemplo de conexão:

```
python
from sqlalchemy import create_engine

engine = create_engine(
    "postgresql+psycopg2://usuario:senha@localhost:5432/minhabase"
)
conn = engine.connect()
```

7. Consulta SELECT:

```
python
from sqlalchemy import text

resultado = conn.execute(text("SELECT * FROM clientes"))

for linha in resultado:
    print(linha)

conn.close()
```

Exemplo usando ORM:

```
python
from sqlalchemy.orm import Session

with Session(engine) as session:
    clientes = session.query(Cliente).all()
    for c in clientes:
        print(c.nome)
```

5. sqlite3

1. Objetivo principal: Biblioteca nativa do Python para trabalhar com o SQLite, um banco de dados leve, local e baseado em arquivo — ideal para projetos pequenos, testes e protótipos.

2. Tipos de banco suportados: Apenas SQLite.

3. Relacional ou não relacional: Relacional.

4. SQL puro, ORM ou ambos: Apenas SQL puro.

5. Instalação: Não é necessária — já vem embutida na biblioteca padrão do Python.

6. Exemplo de conexão:

```
python
import sqlite3

conn = sqlite3.connect("minhabase.db")
```

7. Consulta SELECT:

```
python
cursor = conn.cursor()
cursor.execute("SELECT * FROM clientes")

for linha in cursor.fetchall():
    print(linha)

conn.close()
```

6. pymongo

1. Objetivo principal: Ser o driver oficial para conectar aplicações Python ao MongoDB, permitindo manipular documentos (no formato semelhante a JSON) diretamente do banco.

2. Tipos de banco suportados: Apenas MongoDB.

3. Relacional ou não relacional: Não relacional (NoSQL, orientado a documentos).

4. SQL puro, ORM ou ambos: Nenhum dos dois — não usa SQL, e sim uma API própria orientada a métodos (como `.find()`, `.insert_one()`, etc). Não é considerado um ORM tradicional, pois trabalha com documentos, não com tabelas.

5. Instalação:

```
bash
pip install pymongo
```

6. Exemplo de conexão:

```
python
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["minhabase"]
```

7. Consulta equivalente ao SELECT:

```
python
colecacao = db["clientes"]

# Busca todos os documentos (equivalente a SELECT * FROM clientes)
for documento in colecacao.find():
    print(documento)

# Busca com filtro (equivalente a WHERE)
for documento in colecacao.find({"cidade": "São Paulo"}):
    print(documento)
```

7. redis-py

1. Objetivo principal: Permitir a comunicação com o Redis, um banco de dados em memória, extremamente rápido, geralmente usado para cache, filas, sessões e dados temporários.

2. Tipos de banco suportados: Apenas Redis.

3. Relacional ou não relacional: Não relacional (NoSQL, do tipo chave-valor).

4. SQL puro, ORM ou ambos: Nenhum dos dois — usa comandos próprios do Redis (como **SET**, **GET**, **HSET**), sem SQL e sem ORM.

5. Instalação:

```
bash
pip install redis
```

6. Exemplo de conexão:

```
python
import redis

conn = redis.Redis(
    host="localhost",
```

```
port=6379,  
decode_responses=True  
)
```

7. Consulta equivalente ao SELECT:

```
python  
# Redis não tem "SELECT" como SQL, pois trabalha com chave-valor  
  
# Salvando um dado  
conn.set("cliente:1", "João Silva")  
  
# Buscando um dado (equivalente a consultar um registro)  
valor = conn.get("cliente:1")  
print(valor)  
  
# Buscando várias chaves que combinam com um padrão  
chaves = conn.keys("cliente:*")  
for chave in chaves:  
    print(chave, conn.get(chave))
```